

Molecular dynamics: 2D Lennard-Jones Simulation

Jakob Drusany, Yon Ploj

I. INTRODUCTION

This report presents a parallel implementation of the two-dimensional Lennard-Jones particle simulation. The algorithm is parallelized using CUDA and evaluated on the Arnes HPC cluster across several particle counts and thread configurations.

II. IMPLEMENTATION DETAILS

A. Dividing the work among CUDA threads

The algorithm computes the forces between particles in a pairwise fashion. In a naïve implementation, we would calculate one force (pair of particles) per thread.

In order to utilize shared memory as much as possible, we divided the work into tiles. We delegated each source particle to its own GPU thread, then we loaded a tile of destination particles into shared memory and calculated the forces for each of them. Then, we synchronized the threads in a block and loaded the next tile into shared memory. The hypothesis was, that the overhead of thread synchronization is small enough to benefit from the reduction of individual memory fetches. For simplicity, we fixed the tile size to equal the number of threads per block.

For block level summation reduction, we used tree-like summation; for grid level summation we used `atomicAdd`.

B. Estimating the optimal division

According to documentation, our GPUs (Tesla V100s) have 128kB of shared memory per SM. Because the maximum block size is 2048 and each thread uses $2 * 8B$ of memory (x coordinate and y coordinate), it can only use 32kB, so shared memory will not be a bottleneck.

We decided to estimate the optimal number of threads per block experimentally.

C. Removing the square root

We found that the square root operation is unnecessary for the algorithm to work. Due to the computational complexity of the square root operation, we obtained an additional speedup by adjusting the formulas to avoid calculating the square root.

III. PERFORMANCE ANALYSIS

To accurately assess the speed of our implementation, we ran the experiment with 1000, 2000, 4000, and 8000 particles for 5000 simulation steps.

To estimate the optimal parameter configuration, we additionally tested each configuration against various thread counts. Most were omitted from the tables for brevity.

Table I
AVERAGE SIMULATION TIME (SECONDS) FOR DIFFERENT
NUMBER OF THREADS

Particles	Sequential	1	16	32	64
1000	31.952	3.037	2.485	2.839	2.818
2000	114.093	7.330	3.582	4.001	4.103
4000	428.080	28.296	5.999	6.335	6.317
8000	1658.712	107.943	11.483	11.143	11.381

IV. RESULTS

In table I we see a breakdown of running times for various combinations of particle counts and threads per block.

Firstly, we note that even a single thread per block easily beats the sequential running time.

With the optimal configuration, we achieved a speedup factor as high as 149.

The optimal thread count was observed to be 32.

V. CONCLUSION

This implementation successfully parallelizes the Lennard-Jones algorithm. We achieved a significant speedup compared to sequential computation (by a factor of up to 149).

We achieved the best performance at 32 threads per block.